Machine Learning for Mechanical Engineers at CoEP Session 16: KMeans

Session 16: KMeans

Clustering

The Problem

- Imagine a storekeeper who keeps a record of all his customers' purchase histories.
- This allows him to look up the type of products an individual buyer might be interested in.
- However, doing this for each individual is grossly inefficient.
- A better solution would be to categorize his customers into groups, with each group having similar preferences.
- This would allow him to reach out to more customers with each product recommendation.

The Questions

The problem is, the storekeeper does not know:

- How his customers should be categorized?
- How many of such categories exist?

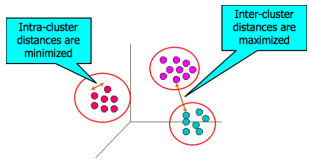
Clustering

So, how do I discover natural groupings or segments in my data?

- Often we are given a large mass of data with no training labels.
- So we have no prior idea what to look for.
- We can get some insight to get started, looking for similar items or "clusters"

Clustering

- Organizing data into classes such that there is
  - High intra-class similarity
  - Low inter-class similarity
- More informally, finding natural groupings among objects.



(Reference: Introduction to Data Mining - Tan, Steinbach, Kumar)

Types of Clustering

Partition vs. Hierarchical

- Partition Clustering: A division of data into non-overlapping clusters, such that each data object is in exactly one subset
- Hierarchical Clustering: A set of nested clusters organized as a hierarchical tree
  - Each node (cluster) is union of its children (sub-clusters)
  - Root of tree: cluster containing all data objects
  - Leaves of tree: singleton clusters

Complete vs. Partial

- Complete Clustering: Every object is assigned to a cluster
- Partial Clustering: Not every object needs to be assigned
  - Motivation: some objects in a dataset may not belong to well-defined groups
  - Noise, outliers, or simply "uninteresting background" data

Exclusive vs. Non-exclusive

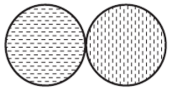
- Exclusive Clustering: Assignment is to one cluster
- Non-Exclusive Clustering: Data objects may belong to multiple clusters
  - Motivation: multi-class situations
  - Example: "student employee"

Well-Separated Clusters: any point in a cluster is closer to every other point in the cluster than to any point not in the cluster

(a) Well-separated clusters. Each point is closer to all of the points in its cluster than to any point in another cluster.

Center-based Clusters

- an object in a cluster is closer to the center of a cluster than to the center of any other cluster
- Center of a cluster ("the most central point"):
  - Centroid: the mean of all the points in the cluster (usually for continuous attributes)
  - Medoid: the most "representative" point of a cluster (usually for categorical attributes)



(b) Center-based clusters. Each point is closer to the center of its cluster than to the center of any other cluster.

Clustering Aspects

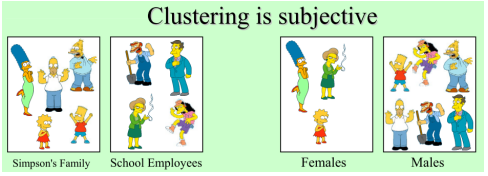
Clustering

Notion of a Cluster can be Ambiguous

(Reference: K means Clustering Algorithm - Kasun Ranga Wijeweera)

Clustering

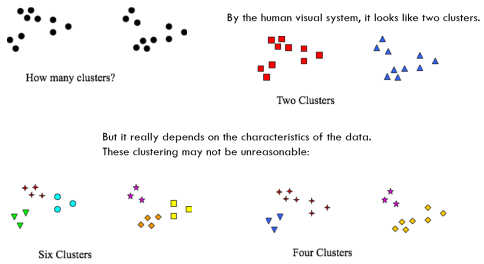
Notion of a Cluster can be Ambiguous



(Reference: K means Clustering Algorithm - Kasun Ranga Wijeweera)

Clustering

Notion of a Cluster can be Ambiguous



K-means

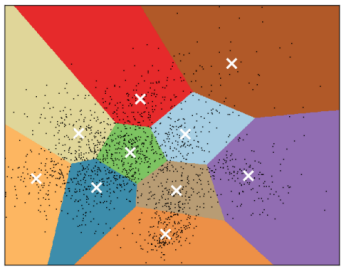
Clustering

- K-Means Clustering is one of the many techniques that constitute "unsupervised learning".
- Creating a labeling of objects with cluster (class) labels
- But, these labels are derived exclusively from the data.

K-Means Clustering

- Formally: a method of vector quantization
- Partition space into Voronoi cells
- Separate samples into n groups of equal variance
- Uses the Euclidean distance metric

K-Means Clustering



Clustering by K-Means

- It operates by computing the "mean" of some attributes.
- That "mean" then becomes the center of one cluster.
- Its procedure follows a simple and easy way to classify a given data set through a certain number of clusters (assume k clusters).
- Data points inside a cluster are homogeneous and heterogeneous to peer groups.

How to cluster?

- Clustering is different than prediction
- Dividing data into groups (clusters) in some meaningful or useful way
- Clustering should capture "natural structure of the data"

Clustering Algorithms

- Clustering is different than prediction
- K-nearest neighbors is very different from K-means, although both are somewhat based on 'Similarity' or 'Distance'
- It is a classification (or regression) algorithm that in order to determine the classification of a point,
- Decides class of the test case based on the classes of the K nearest points.
- It is supervised because you are trying to classify a point based on the known classification of other points.)

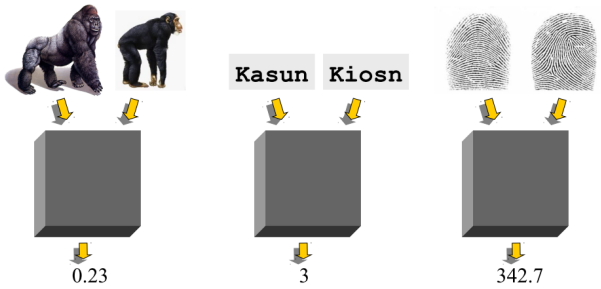
How to cluster?

- How to measure "proximity"?
- Proximity: similarity or dissimilarity between two objects
- Similarity: numerical measure of the degree to which two objects are alike

Proximity - Distance Measures

Defining Distance Measure/MetricS

Say, some black box calculations.



Some distance formulas. (Reference: K means Clustering Algorithm - Kasun Ranga Wijeweera)

Dissimilarities between Data Objects

- A common measure for the proximity between two objects is the Euclidean Distance
- Defined for one-dimension, two-dimensions, three-dimensions, any n-dimensional space
- Generically: Minkowski Distance Metric
  - $r = 1$: L_1 norm: Manhattan or Taxi cab distance
  - $r = 2$: L_2 norm: Euclidean distance

Intuition

K-means uses Euclidean (L_2) distance by default, which means it implicitly assumes clusters are roughly spherical and that all features are on comparable scales – a feature with a much larger numeric range will dominate the distance calculation and effectively decide the clustering on its own, so features usually need to be standardized first.

Example

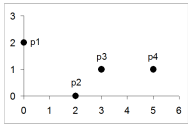
Table: The speed and agility ratings for 20 college athletes labelled with the decisions for whether they were drafted or not.

ID	Speed	Agility	Draft	ID	Speed	Agility	Draft
1	2.50	6.00	No	11	2.00	2.00	No
2	3.75	8.00	No	12	5.00	2.50	No
3	2.25	5.50	No	13	8.25	8.50	No
4	3.25	8.25	No	14	5.75	8.75	Yes
5	2.75	7.50	No	15	4.75	6.25	Yes
6	4.50	5.00	No	16	5.50	6.75	Yes
7	3.50	5.25	No	17	5.25	9.50	Yes
8	3.00	3.25	No	18	7.00	4.25	Yes
9	4.00	4.00	No	19	7.50	8.00	Yes
10	4.25	3.75	No	20	7.25	5.75	Yes

$d(id_{12}, id_5) = \sqrt{(5.00 - 2.75)^2 + (2.50 - 7.50)^2} = \sqrt{30.0625} = 5.4829$

Distance Matrix

- Once a distance metric is chosen, the proximity between all of the objects in the dataset can be computed
- Can be represented in a distance matrix
- Pairwise distances between points



L1 norm distance. "Manhattan" Distance.

L1	p1	p2	p3	p4
p1	0	4	4	6
p2	4	0	2	4
p3	4	2	0	2
p4	6	4	2	0

L2 norm distance. Euclidean Distance.

L2	p1	p2	p3	p4
p1	0	2.828	3.162	5.099
p2	2.828	0	1.414	3.162
p3	3.162	1.414	0	2
p4	5.099	3.162	2	0

K-means Algorithm

How K-means forms cluster?

Algorithm

- Iterative refinement
- Three basic steps
- Step 1: Choose k
- Iterate over:
  - Step 2: Assignment
  - Step 3: Update centroids
- Repeats until convergence has been reached

How K-means forms cluster?

Algorithm

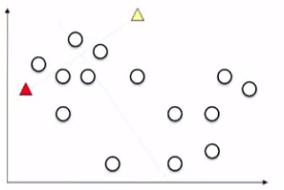
1. K-means picks k points known as centroids.
2. Each data point finds distances with each centroid and goes to the one who is closest.
3. In the next iteration, it computes a new centroid of each cluster based on the latest cluster members.
4. As we have new centroids, repeat step 2 and 3.
5. Repeat this process until convergence occurs i.e. centroids do not change.

Intuition

Each of the two alternating steps can only ever improve or maintain the current clustering: reassigning points to their nearest centroid can't increase total distance, and recomputing a centroid as the mean of its points minimizes distance to exactly those points. Since the quantity being optimized never gets worse and there are only finitely many ways to partition the data, the process is guaranteed to stop – though not necessarily at the best possible clustering, only a locally good one.

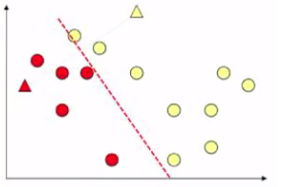
Algorithm: Steps

- Data points are K=2 given.
- So random two points are chosen as centroids



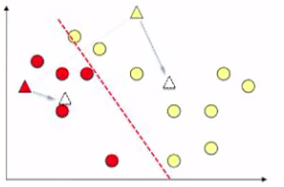
Algorithm: Steps

- Each data point is assigned to closest centroid
- Closeness is based on Distance



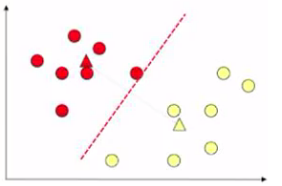
Algorithm: Steps

- Calculate new centroids of respective groups
- By averaging NUMERIC variables only.



Algorithm: Steps

- Repeat.
- Memberships may change now.
- Update till centroids don't move appreciably.
- Clustering is DONE.



K-Means Implementation

K-means Algorithm : From Scratch

Load data

```
import numpy as np
from scipy.io import loadmat

data = loadmat('data/ex7data2.mat')
X = data['X']
```

K-means Algorithm : From Scratch

A function that finds the closest centroid for each instance in the data:

```
def find_closest_centroids(X, centroids):
    m = X.shape[0]
    k = centroids.shape[0]
    idx = np.zeros(m)

    for i in range(m):
        min_dist = 1000000
        for j in range(k):
            dist = np.sum((X[i,:] - centroids[j,:]) ** 2)
            if dist < min_dist:
                min_dist = dist
                idx[i] = j

    return idx
```

K-means Algorithm : From Scratch

Test

```
initial_centroids = np.array([[3, 3], [6, 2], [8, 5]])

idx = find_closest_centroids(X, initial_centroids)

idx[0:3]
array([ 0.,  2.,  1.])
```

K-means Algorithm : From Scratch

A function to compute the centroid of a cluster:

```
def compute_centroids(X, idx, k):
    m, n = X.shape
    centroids = np.zeros((k, n))

    for i in range(k):
        indices = np.where(idx == i)
        centroids[i,:] = (np.sum(X[indices,:], axis=1) /
                 len(indices[0])).ravel()
    return centroids

compute_centroids(X, idx, 3)
array([[ 2.42830111,  3.15792418],
       [ 5.81350331,  2.63365645],
       [ 7.11938687,  3.6166844 ]])
```

K-means Algorithm : From Scratch

We can init centroids using some random sample of data-points as well

```
def init_centroids(X, k):
    m, n = X.shape
    centroids = np.zeros((k, n))
    idx = np.random.randint(0, m, k)
    for i in range(k):
        centroids[i,:] = X[idx[i],:]
    return centroids

init_centroids(X, 3)
array([[ 1.15354031,  4.67866717],
       [ 6.27376271,  2.24256036],
       [ 2.20960296,  4.91469264]])
```

K-means Algorithm : From Scratch

Core

```
def run_k_means(X, initial_centroids, max_iters):
    m, n = X.shape
    k = initial_centroids.shape[0]
    idx = np.zeros(m)
    centroids = initial_centroids

    for i in range(max_iters):
        idx = find_closest_centroids(X, centroids)
        centroids = compute_centroids(X, idx, k)

    return idx, centroids
```

K-means Algorithm : From Scratch

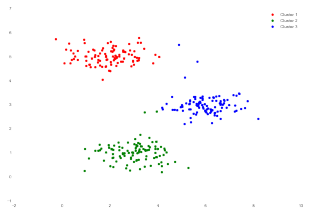
Run

```
idx, centroids = run_k_means(X, initial_centroids, 10)

cluster1 = X[np.where(idx == 0)[0],:]
cluster2 = X[np.where(idx == 1)[0],:]
cluster3 = X[np.where(idx == 2)[0],:]
```

K-means Algorithm : From Scratch

Plot:



Code:

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(12,8))
ax.scatter(cluster1[:,0], cluster1[:,1], s=30, color='r',
      label='Cluster 1')
ax.scatter(cluster2[:,0], cluster2[:,1], s=30, color='g',
      label='Cluster 2')
ax.scatter(cluster3[:,0], cluster3[:,1], s=30, color='b',
      label='Cluster 3')
ax.legend()
```

Summary

Algorithm: Summary

- Input: K, set of points $x_1 \dots x_n$
- Place centroids $c_1 \dots c_K$ at random locations
- Repeat until convergence:
  - for each point x_i:
    - find nearest centroid c_j $\arg\min_j D(x_i, c_j)$ (distance (e.g. Euclidian) between instance x_i and cluster center c_j)
    - assign the point x_i to cluster j
  - for each cluster j = 1 ... K: $c_j(a) = \frac{1}{n_{j,\text{new}}} \sum_{x_i \in \text{new}_j} x_i(a)$ for $a = 1..d$
    - new centroid c_j = mean of all points x_i assigned to cluster j in previous step
- Stop when none of the cluster assignments change

O (#iterations * #clusters * #instances * #dimensions)

(Note: Only numeric attributes can be averaged for centroid calculations)

(Reference: K Means Clustering - Victor Lavrenko)

K-means

- Advantages
  - Scales well
  - Efficient
- Disadvantages
  - Choosing the wrong k

K-means

When to use?

- Normally distributed data
- Large number of samples
- Not too many clusters
- Distance can be measured in a linear fashion

Quick Check: K-means

Think About It

If you run k-means twice on the exact same dataset with the same k, but starting from different random initial centroids, could you get two different final clusterings? What does your answer imply about how many times you should run k-means in practice?

Quick Check: K-means

Answer

Yes – k-means only guarantees convergence to a local optimum of the within-cluster distance, not the global one, and which local optimum it lands in depends on where the centroids started. This is why implementations like scikit-learn's `KMeans` run the algorithm several times with different random initializations and keep the best result, rather than trusting a single run.

Applications

- Business
  - Businesses collect large amounts of information on current and potential customers.
  - Clustering to segment customers into a small number of groups, for additional analysis and marketing activities
- Clustering for Utility
  - Efficiently Finding Nearest Neighbors
  - Alternative to computing the pairwise distance between all points